

Object-Oriented HALs for Microcontrollers: How IOsonata Compares

Framework	OO Design Depth	Performance	Complexity/Overhead
Vendor HAL (C only)	None (procedural only)	Baseline (often slower)	Low complexity, but messy macros
Zephyr / RTOS	Partial (C structs, some C++)	High overhead (latency, memory)	Complex, steep learning curve
modm (C++)	Strong OO (templates, drivers)	Moderate (templates add bloat)	Complex build system
QP Framework	OO State Machines	Efficient in events, not HAL focused	Good for RTOS-like design
IOsonata	Full OO polymorphic HAL (Device, Intrf, Sensors)	Faster than vendor SDK (benchmarked)	Lean, clean, minimal overhead